

SIMATS ENGINEERING

Short Answer Test

COURSE NAME: SOFTWARE TESTING

COURSE CODE:

CSA37 COURSE OUTCOME COVERED

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3

8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3

19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3



Code of Conduct:

I AATHISRI(192521252) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

AATHISRI
192521252

1. TESTING AS AN ENGINEERING ACTIVITY

Software testing should be treated as a **systematic and engineering-based activity** rather than random testing.

- Testing should begin with **test planning**, where objectives, scope, resources, and risks are identified.
- Test cases should be designed systematically using techniques such as **Equivalence Partitioning, Boundary Value Analysis, Decision Tables, and State Transition Testing**.
- Each test case should have defined **inputs, expected results, and actual results**.
- Structured testing provides better **test coverage** and avoids missing important scenarios.
- It also makes defect detection more effective and testing results more consistent.

Example: For an age field accepting values from 18 to 60, boundary testing can check 17, 18, 60, and 61 to detect boundary-related defects.

Conclusion: Structured test design improves software quality by providing systematic coverage and detecting defects that random testing may miss.

2. WELL-DEFINED TESTING PROCESS IN A BANKING SYSTEM

A banking system requires a **well-defined testing process** because failures can cause financial loss and security problems.

- The process begins with **test planning and requirement analysis**.
- Test cases are designed for functional, security, performance, and reliability requirements.
- Testing is performed at different levels such as **unit, integration, system, and acceptance testing**.
- Defects are recorded, fixed, retested, and followed by **regression testing**.
- A standardized process ensures consistent testing and improves system reliability.

Example: For an ATM withdrawal, testing should include valid PIN, wrong PIN, insufficient balance, invalid amount, network failure, and successful withdrawal.

Conclusion: A defined testing process reduces failures, improves reliability, and ensures that critical banking operations work correctly.

3. TESTING AS A STRUCTURED PROCESS IN SDLC

Testing should be performed as a **continuous and structured process throughout the SDLC**, rather than only at the end.

- During the **requirements phase**, testers review requirements and identify ambiguities.
- During the **design phase**, test conditions and test scenarios are prepared.
- During **coding**, developers perform unit testing.
- During integration, **integration testing** verifies communication between modules.
- During the system phase, **system and acceptance testing** verify the complete application.
- Regression testing is performed after modifications to ensure existing functions are not affected.

Example: In an e-commerce system, the payment module must be tested with the shopping cart, order management,

and database before final system testing.

Conclusion: Process discipline provides continuous defect detection, better coverage, and higher-quality software.

4. ERROR, DEFECT, AND FAILURE

Error, defect, and failure are related but represent different stages of a software problem.

- **Error:** A human mistake made during requirements, design, coding, or testing.
- **Defect:** A fault in the software caused by an error.
- **Failure:** The incorrect behavior observed when the software executes the defective code.
- An error does not always produce a failure because the defective code may never be executed.
- The relationship can be represented as **Error** → **Defect** → **Failure**.

Example:

A developer misunderstands the password requirement (**error**). The login program is coded incorrectly (**defect**). When a user enters an incorrect password and is still allowed to log in (**failure**).

Conclusion: Understanding these terms helps testers correctly identify, report, and analyze software problems.

5. PRINCIPLE: TESTING EARLY

The testing principle “**Testing early saves time and money**” states that defects should be detected as early as possible in the SDLC.

- Testers should participate during the **requirements and design phases**.
- Early reviews can identify missing, incorrect, or ambiguous requirements.
- Defects found early are generally **easier and cheaper to fix**.
- Early testing prevents defects from being passed to later development stages.
- It also improves overall test effectiveness and reduces rework.

Example: If a payment requirement is found to be ambiguous during requirement review, it can be corrected immediately. If discovered after deployment, it may require changes to design, code, database, and testing.

Conclusion: Early testing reduces development cost, prevents defect propagation, and improves software quality.

6. TESTER CONTRIBUTION DURING THE REQUIREMENT PHASE

Testers can contribute significantly during the **requirement analysis phase** by reviewing requirements before development begins.

- Testers identify **ambiguous, incomplete, inconsistent, or missing requirements**.
- They check whether each requirement is **clear, measurable, and testable**.
- Testers can prepare initial test scenarios and acceptance criteria from the requirements.
- They can identify possible **risks and exceptional conditions** early.

- Early involvement prevents defects from entering the design and implementation stages.

Example: In a healthcare system, a requirement such as “the system should respond quickly” is ambiguous. A tester can request a measurable requirement such as “the system should display patient records within 2 seconds.”

Importance: Early tester involvement improves requirement quality, reduces rework, lowers defect-fixing costs, and increases confidence in the final system.

7. DEFECTS CAUSED BY UNCLEAR REQUIREMENTS

Defects can originate at the **requirement stage** and propagate through the entire software lifecycle.

- If requirements are **unclear, incomplete, or ambiguous**, developers may interpret them incorrectly.
- Incorrect interpretation leads to defects in **design and implementation**.
- Testers may also create incorrect test cases if the expected behavior is not clearly defined.
- These defects may remain undetected until system testing or production, making them expensive to fix.
- Clear requirements should be **complete, consistent, unambiguous, and testable**.

Example: If a payroll requirement does not clearly specify how overtime is calculated, developers may implement the wrong calculation, resulting in incorrect employee salaries.

Conclusion: Clear and testable requirements prevent defects at their source and reduce rework, cost, and failures throughout the software lifecycle.

8. BOUNDARY VALUE ANALYSIS AND BOUNDARY DEFECT

The crash at boundary input values can be classified as a **boundary-related functional defect**. It indicates that boundary conditions were not adequately tested.

- Boundary Value Analysis (BVA) focuses on values at and around the limits of an input range.
- Typical values tested are **minimum, minimum+1, maximum-1, maximum, and values just outside the range**.
- If boundary values are not tested, defects such as crashes, incorrect calculations, or invalid outputs may remain undetected.
- BVA would have exposed the defect before the system was released.
- It improves test coverage with a relatively small number of carefully selected test cases.

Example: If an age field accepts 18–60, test values should include **17, 18, 19, 59, 60, and 61**.

Conclusion: BVA helps identify boundary-related defects early and improves the reliability of input validation.

9. DEFECT REPOSITORY AND TEST CASE DESIGN

A **defect repository** stores information about previously identified defects, including their causes, affected modules, severity, and corrective actions.

- Testers can analyze historical defects to identify **frequently failing areas**.
- Previous defects can be converted into new or improved test cases.

- Similar defects can be included in **regression test suites** for future releases.
- Defect patterns help testers improve test coverage and focus on high-risk areas.
- This prevents the same defects from repeatedly appearing in different releases.

Example: If previous releases repeatedly had payment-calculation defects, testers can create additional boundary, negative, and integration test cases for payment processing.

Conclusion: A defect repository acts as a knowledge base that supports better test design, prevents recurrence, and improves software quality.

10. FUNCTIONAL DEFECT IN AN ATM SYSTEM

An ATM withdrawal failure caused by incorrect transaction logic is a **functional defect** because the system does not perform a required function correctly.

- Functional testing verifies whether the software behaves according to specified requirements.
- Withdrawal scenarios should include valid withdrawal, insufficient balance, invalid amount, and account balance updates.
- The defect could have been prevented through **requirement-based, functional, and integration testing**.
- Negative test cases should also verify incorrect or exceptional transaction conditions.
- Regression testing should be performed after fixing transaction-related defects.

Example: If ₹5,000 is withdrawn but the account balance is reduced by ₹10,000, the transaction logic contains a functional defect.

Conclusion: Proper functional and transaction-based testing would help detect such defects before deployment.

11. DEFECT CLUSTERING

Defect clustering is the testing principle that a small number of modules often contain a large proportion of defects.

- The testing team should analyze historical defect data to identify **high-defect modules**.
- These modules should receive more testing effort and greater test coverage.
- High-risk areas can be tested using additional functional, integration, regression, and negative test cases.
- Low-defect modules can receive comparatively less intensive testing based on risk.
- This helps optimize limited testing time and resources.

Example: If 70% of defects are found in the payment and checkout modules, these modules should receive greater testing priority.

Conclusion: Defect clustering helps testers focus on high-risk areas, improving testing efficiency and defect detection.

12. CLASSIFICATION OF A PERFORMANCE DEFECT

Slow performance under heavy user load is classified as a **performance defect**.

- Performance testing evaluates response time, throughput, scalability, and resource usage.
- Load testing determines how the system behaves under expected numbers of users.
- Stress testing checks behavior beyond normal capacity.
- Classifying the defect correctly helps the team select the appropriate testing and debugging techniques.
- It also helps assign the problem to the appropriate technical team for resolution.

Example: If a website normally responds in 2 seconds but takes 15 seconds when thousands of users access it simultaneously, it is a performance/scalability defect.

Conclusion: Proper defect classification speeds up analysis, prioritization, and resolution.

13. USING HISTORICAL DEFECT DATA FOR REGRESSION TESTING

Historical defect data can be effectively reused to create a **risk-based regression test suite**.

- Identify modules and features where defects occurred frequently in previous releases.
- Convert important historical defects into permanent regression test cases.
- Execute these tests after every major change or release.
- Give higher priority to critical and frequently failing areas.
- Analyze new defect trends and continuously update the regression suite.

Example: If previous releases had repeated defects in login and payment, these areas should always be included in regression testing.

Conclusion: Reusing historical defect data improves test coverage, reduces repeated defects, and saves testing time by focusing on high-risk areas.

14. TESTER'S ROLE IN MINIMIZING INTEGRATION DEFECTS

Testers play an important role in identifying and preventing **integration defects** caused by poor communication between development and testing teams.

- Testers should participate early in requirements and design discussions.
- They should communicate integration risks and clarify expected behavior with developers.
- Test scenarios and interface requirements should be discussed collaboratively.
- Defects should be reported with clear steps, evidence, expected results, and actual results.
- Regular developer-tester collaboration helps identify problems before integration testing.

Example: If the developer assumes a date format is DD/MM/YYYY while the tester expects MM/DD/YYYY, early communication can prevent integration failures.

Conclusion: Early tester involvement and continuous communication improve integration quality and reduce misunderstandings.

15. DEFECT PREVENTION FOR RECURRING DEFECTS

When similar defects appear repeatedly, the team should focus on **defect prevention** rather than only fixing individual defects.

- Analyze recurring defects using **root cause analysis**.
- Identify whether the cause is poor requirements, design, coding practices, or inadequate testing.
- Improve development and testing processes based on the root cause.
- Introduce reviews, coding standards, automated tests, and additional regression tests where required.
- Track defect trends to confirm whether preventive actions are effective.

Example: If incorrect input validation repeatedly causes defects, the team can introduce coding standards and reusable validation components.

Conclusion: Defect prevention eliminates the underlying causes of defects, reducing recurrence and improving long-term software quality.

16. STANDARDIZED TESTING STRATEGY FOR A MOBILE APPLICATION

A mobile application with frequent crashes requires a **systematic and standardized testing approach**.

- Define a common test process including **planning, test design, execution, defect reporting, and regression testing**.
- Use standard test techniques such as equivalence partitioning and boundary value analysis.
- Test across different devices, operating systems, screen sizes, and network conditions.
- Maintain reusable test cases and regression suites.
- Track defects and use historical data to improve future testing.

Example: The application should be tested on different supported Android/iOS versions and under conditions such as weak network connectivity.

Conclusion: A structured testing approach provides consistent coverage, detects more defects, and improves application reliability.

17. TESTING VS DEBUGGING

Testing and debugging are different activities, although both contribute to software quality.

Testing

Finds and reports defects.

Usually performed by testers and developers.

Compares actual results with expected results.

Does not necessarily identify the exact cause.

Debugging

Finds and fixes the cause of a defect.

Mainly performed by developers.

Examines code, logic, and program execution.

Identifies and corrects the root cause.

Example: In an online banking system, testing discovers that the balance is incorrectly updated after a transfer. Debugging then examines the transaction code and corrects the calculation.

Conclusion: Testing detects failures and defects, while debugging locates and fixes their causes. Both are necessary for software quality.

18. CROSS-BROWSER TESTING STRATEGY

Cross-browser testing ensures that a web application works consistently across different **browsers, versions, operating systems, and devices**.

- Identify the browsers and versions that are officially supported.
- Prepare common test cases for important functions such as login, payment, navigation, and forms.
- Execute tests on browsers such as Chrome, Firefox, Edge, and Safari as applicable.
- Compare functionality, layout, compatibility, and performance across browsers.
- Record browser-specific defects and perform regression testing after fixes.

Example: If a checkout button works correctly in Chrome but does not respond in Safari, the issue should be recorded as a browser compatibility defect.

Conclusion: A planned cross-browser testing strategy improves compatibility, user experience, and customer satisfaction.

19. IMPROPER VALIDATION IN A LOGISTICS APPLICATION

Improper validation can cause a logistics system to produce **inconsistent or incorrect outputs**. Testing should verify both valid and invalid inputs.

- Identify all input fields and define their **valid ranges, formats, and constraints**.
- Use techniques such as **Equivalence Partitioning and Boundary Value Analysis**.
- Test invalid, missing, duplicate, and unexpected inputs.
- Perform integration testing to ensure validated data remains correct between modules.
- Use regression testing after fixing validation defects.

Example: If a shipment quantity must be between 1 and 100, test 0, 1, 2, 99, 100, and 101.

Conclusion: Proper validation testing detects errors early, prevents incorrect data processing, and improves system reliability.

20. TESTING AS AN ENGINEERING ACTIVITY

Random testing provides limited and inconsistent coverage. Testing should therefore be treated as a **systematic engineering activity**.

- Start with test planning based on requirements and risks.
- Design test cases systematically using techniques such as **Equivalence Partitioning, Boundary Value**

Analysis, and Decision Tables.

- Define expected results for every test case.
- Maintain traceability between requirements and test cases.
- Analyze test results and perform regression testing after fixes.

Example: Instead of randomly testing a login page, test valid credentials, invalid username, invalid password, empty fields, account lockout, and boundary conditions.

Conclusion: Systematic test design improves coverage, reduces missed defects, and makes testing repeatable and measurable.

21. DATA INCONSISTENCY DURING INTEGRATION

Data inconsistency between healthcare modules is usually an **integration defect** caused by incorrect communication, data formats, interfaces, or synchronization.

- Review the interfaces and data flow between modules.
- Verify that the same data has consistent **format, type, and meaning** across modules.
- Perform integration testing using valid and invalid data.
- Check database updates and transaction handling after data is transferred.
- Use end-to-end and regression testing to ensure fixes do not create new problems.

Example: If a patient's date of birth is stored as DD/MM/YYYY in one module but interpreted as MM/DD/YYYY in another, integration testing can expose the inconsistency.

Conclusion: Proper integration testing verifies data flow between modules and ensures accurate and consistent information throughout the system.

22. ROLE OF TESTERS ACROSS SDLC PHASES

Testers should participate throughout the **Software Development Life Cycle**, rather than joining only during final testing.

- **Requirements:** Review requirements and identify ambiguity, missing conditions, and risks.
- **Design:** Review designs and prepare test conditions and scenarios.
- **Development:** Support unit/integration testing and clarify expected behavior.
- **System testing:** Execute functional, performance, security, and other required tests.
- **Maintenance:** Perform regression testing after changes and fixes.

Example: A tester identifying an unclear payment requirement during analysis can prevent incorrect implementation.

Conclusion: Early tester involvement improves communication, detects defects earlier, reduces rework, and improves overall software quality.

23. TEST COVERAGE FOR PEAK USAGE FAILURES

A system failing during peak usage indicates insufficient **test coverage**, particularly for performance and high-load scenarios.

- Identify critical usage scenarios and expected maximum user load.
- Perform **load, stress, and scalability testing**.
- Test concurrent users, high transaction volumes, and resource-intensive operations.
- Include realistic peak and abnormal usage conditions in test cases.
- Analyze performance results and fix bottlenecks before deployment.

Example: An online ticketing system should be tested with thousands of simultaneous users before a major event.

Conclusion: Better test coverage includes normal, peak, and exceptional conditions, reducing production failures and improving system reliability.

24. DEFECT REPOSITORY AND DEFECT MANAGEMENT

A **defect repository** provides a central location for recording and tracking software defects throughout their lifecycle.

- Record details such as defect ID, description, severity, priority, environment, steps to reproduce, and status.
- Assign defects to responsible team members and track them from **New** → **Assigned** → **Fixed** → **Retested** → **Closed**.
- Maintain evidence such as screenshots, logs, or test data where appropriate.
- Analyze historical defects to identify recurring problems and high-risk modules.
- Use the information to improve test cases and regression testing.

Example: If the same payment defect occurs repeatedly, repository data can help identify its root cause and add specific regression tests.

Conclusion: Good defect documentation improves traceability, communication, analysis, and long-term software quality.

25. STRUCTURED TESTING FOR A BANKING SYSTEM

Repeated transaction failures indicate that the banking system lacks **systematic and comprehensive testing**.

- Begin with clear test planning based on banking requirements and risks.
- Perform testing at different levels: **unit, integration, system, and acceptance testing**.
- Include positive, negative, boundary, security, and transaction-related scenarios.
- Perform regression testing whenever transaction logic is changed.
- Use defect history to focus testing on frequently failing modules.

Example: A fund-transfer feature should test successful transfers, insufficient balance, invalid accounts, duplicate transactions, network failures, and incorrect amounts.

Conclusion: Structured testing improves transaction reliability, reduces repeated failures, and increases confidence in critical banking operations.

26. EDGE CASES AND BOUNDARY TESTING

Testing only normal inputs can miss defects at the **edges or limits** of the system.

- Identify the minimum and maximum allowed input values.
- Test values just below, at, and just above each boundary.
- Include empty, null, maximum-length, and unusual inputs where applicable.
- Use **Boundary Value Analysis** to select important edge cases.
- Add discovered edge cases to the regression suite.

Example: If a system accepts an amount from ₹100 to ₹10,000, test ₹99, ₹100, ₹101, ₹9,999, ₹10,000, and ₹10,001.

Conclusion: Boundary testing provides effective defect detection and ensures that the system handles limits correctly.

27. PROCESS-BASED TESTING AND STANDARDIZATION

Inconsistent testing across modules can be reduced by adopting a **standard testing process**.

- Define common testing procedures, templates, standards, and documentation.
- Use a consistent process for **test planning, design, execution, defect reporting, and closure**.
- Apply common test design techniques across modules.
- Maintain reusable test cases and regression suites.
- Monitor test results and coverage to identify quality gaps.

Example: Every module can follow the same process of requirement review → test case design → execution → defect reporting → retesting → regression testing.

Conclusion: Standardization makes testing consistent, measurable, repeatable, and easier to manage, resulting in more uniform software quality.

28. INPUT VALIDATION AND ROBUSTNESS

Unexpected inputs can cause application failures when proper validation is missing.

- Identify all possible valid and invalid input conditions.
- Test **empty, incorrect, out-of-range, special, duplicate, and unexpected inputs**.
- Apply Equivalence Partitioning and Boundary Value Analysis.
- Verify that invalid inputs generate appropriate error messages instead of causing crashes.
- Perform regression testing after validation fixes.

Example: A registration form should reject an invalid email format, missing required fields, and values outside the permitted age range.

Conclusion: Strong input validation testing makes applications more robust, reliable, and resistant to unexpected user inputs.

29. REDUCING DEFECT LEAKAGE INTO PRODUCTION

Defect leakage occurs when defects are not detected during testing and reach the production environment.

- Analyze production defects to identify their **root causes**.
- Improve requirement reviews and test coverage.
- Use risk-based testing to focus on critical functionality.
- Perform regression, integration, system, and acceptance testing before release.
- Use defect history to improve test cases and prevent repeated leakage.

Example: If payment defects frequently reach production, increase payment integration, negative, boundary, and regression testing before every release.

Conclusion: Stronger test planning, better coverage, and analysis of historical defects reduce production defects and improve software quality.

30. EARLY DEFECT DETECTION AND TESTING STRATEGY

Poor testing strategies often allow defects to remain until later development stages, increasing cost and effort.

- Involve testers during **requirements and design** reviews.
- Apply early reviews and static testing to identify defects before execution.
- Design tests based on requirements, risks, and critical functionality.
- Perform testing continuously at unit, integration, system, and acceptance levels.
- Track defects and use historical information to improve future testing.

Example: Detecting an incorrect requirement during analysis is much easier and cheaper than discovering the same problem after coding and deployment.

Conclusion: Early defect detection reduces rework and maintenance costs, saves time, and improves overall software development efficiency.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand